
Funkcje skrótu

Anna Grocholewska-Czuryło

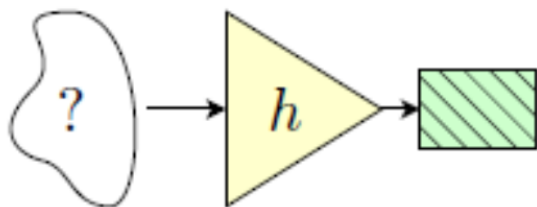
Jednokierunkowe funkcje skrótu

- Argumentem jednokierunkowej funkcji skrótu $H(M)$ jest wiadomość M o dowolnej długości.
- Wartością tej funkcji jest liczba h o ustalonej długości.
 $h=H(M)$, przy czym h jest liczbą o długości m

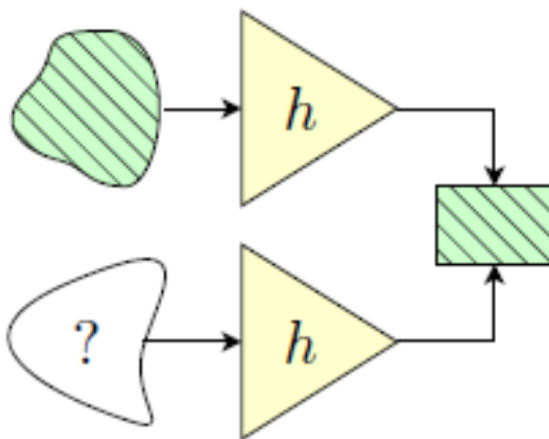
Własności funkcji skrótu

- własność kompresji
- własność łatwości obliczeń, mając dane M łatwo jest obliczyć h
- własność jednokierunkowości, mając dane h , trudno jest obliczyć M (**odporność na przeciwobraz**) - **nieodwracalność**
- własność słabej odporności na kolizje, mając dane M , trudno jest znaleźć inną wiadomość M' taką, że $H(M)=H(M')$ (słabo jednokierunkowa funkcja skrótu) (**odporność na zastępczy przeciwobraz**)
- własność odporności na kolizje jest obliczeniowo trudne znalezienie dwóch dowolnych argumentów $M \neq M'$, dla których $H(M) = H(M')$ (silnie jednokierunkowa funkcja skrótu) (**odporność na kolizje**)

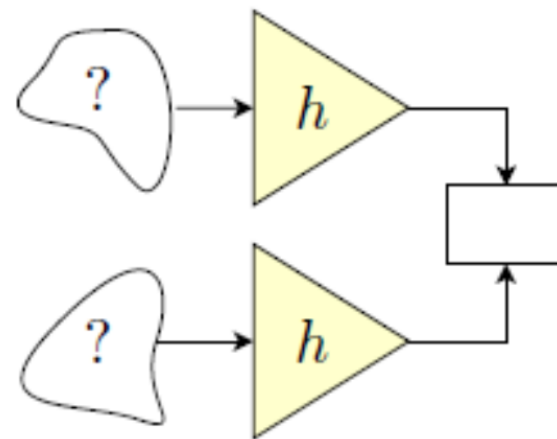
nieodwracalność



słaba bezkolizyjność

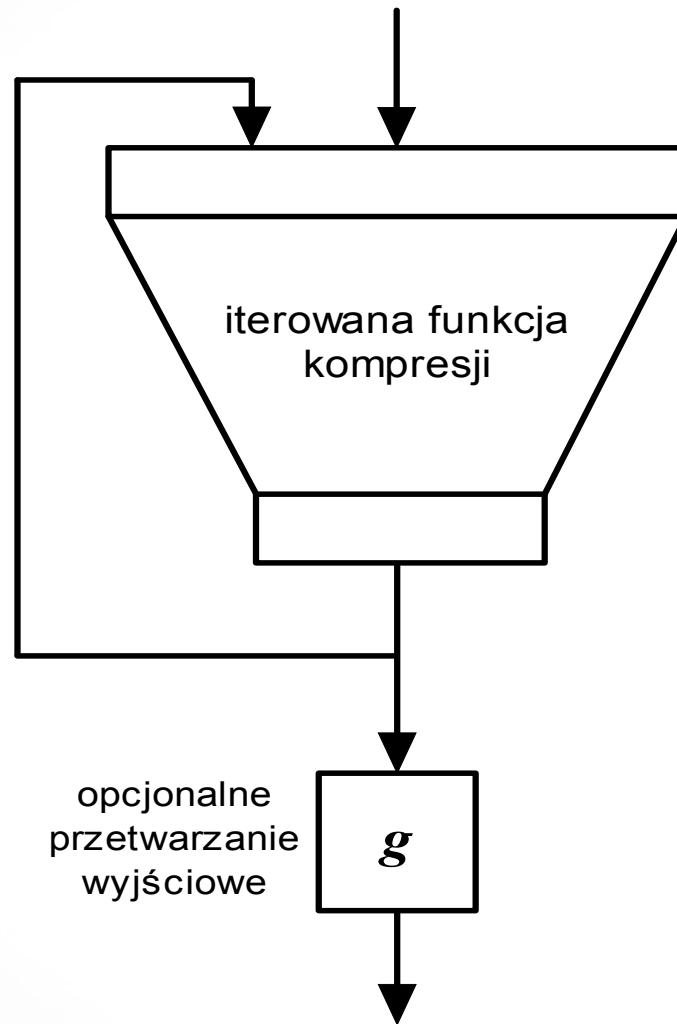


silna bezkolizyjność



Źródło: Przemysław Rodwald, AMW.

wiadomość m o dowolnej długości



skrót H o ustalonej długości

wiadomość m



przetwarzanie
wstępne

uzupełnienie bitami do wymaganej
pełnej wielokrotności

dołączenie informacji o długości
wiadomości

$$m = B_0 B_1 \dots B_{t-1}$$

przetwarzanie
iterowane

B_{i-1}

H_{i-1}

$H_0 = IV$



H_i

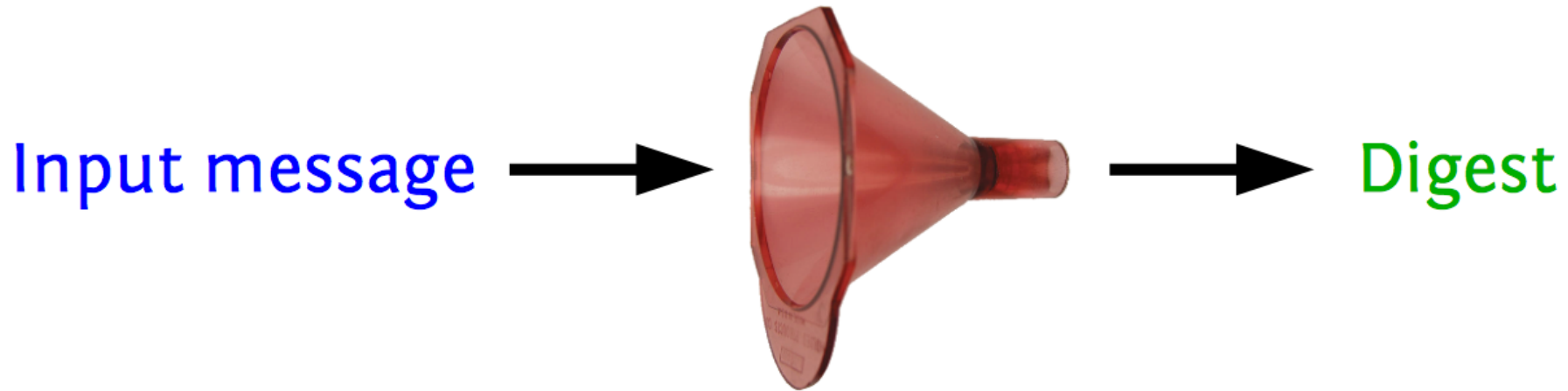
H_t

opcjonalne
przetwarzanie
wyjściowe

g

skrót $H = h(m) = g(H_t)$

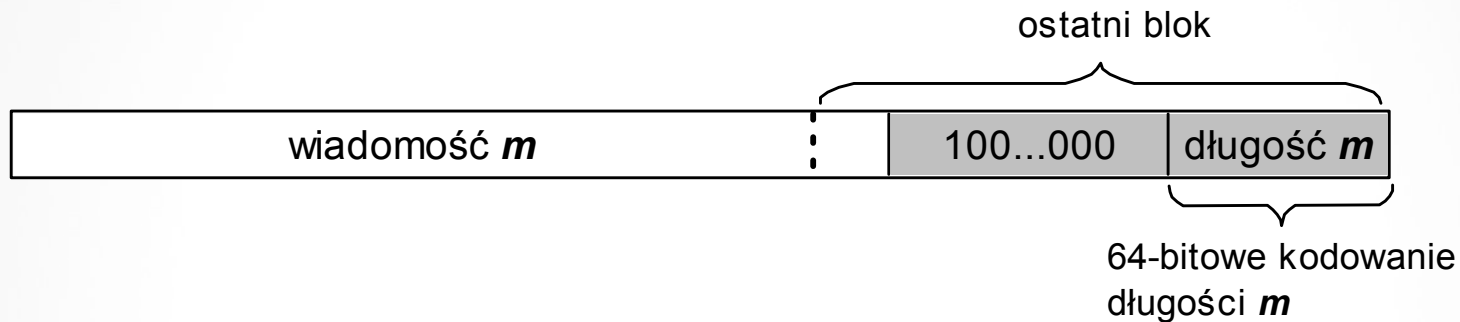
$$h : \{0, 1\}^* \rightarrow \{0, 1\}^n$$



- MD5: $n = 128$ (Ron Rivest, 1992)
- SHA-1: $n = 160$ (NSA, NIST, 1995)
- SHA-2: $n \in \{224, 256, 384, 512\}$ (NSA, NIST, 2001)

Dwie klasy funkcji skrótu

- MDC - Manipulation Detection Code - funkcje skrótu bez klucza kryptograficznego
 - OWHF - odporne na przeciwobraz oraz zastępczy przeciwobraz
 - CRHF - odporne na kolizje (dodatkowo oprócz odporności na zastępczy przeciwobraz)
- MAC – Message Authentication Code - funkcje skrótu z kluczem



różnice w budowie i działaniu funkcji kompresji f

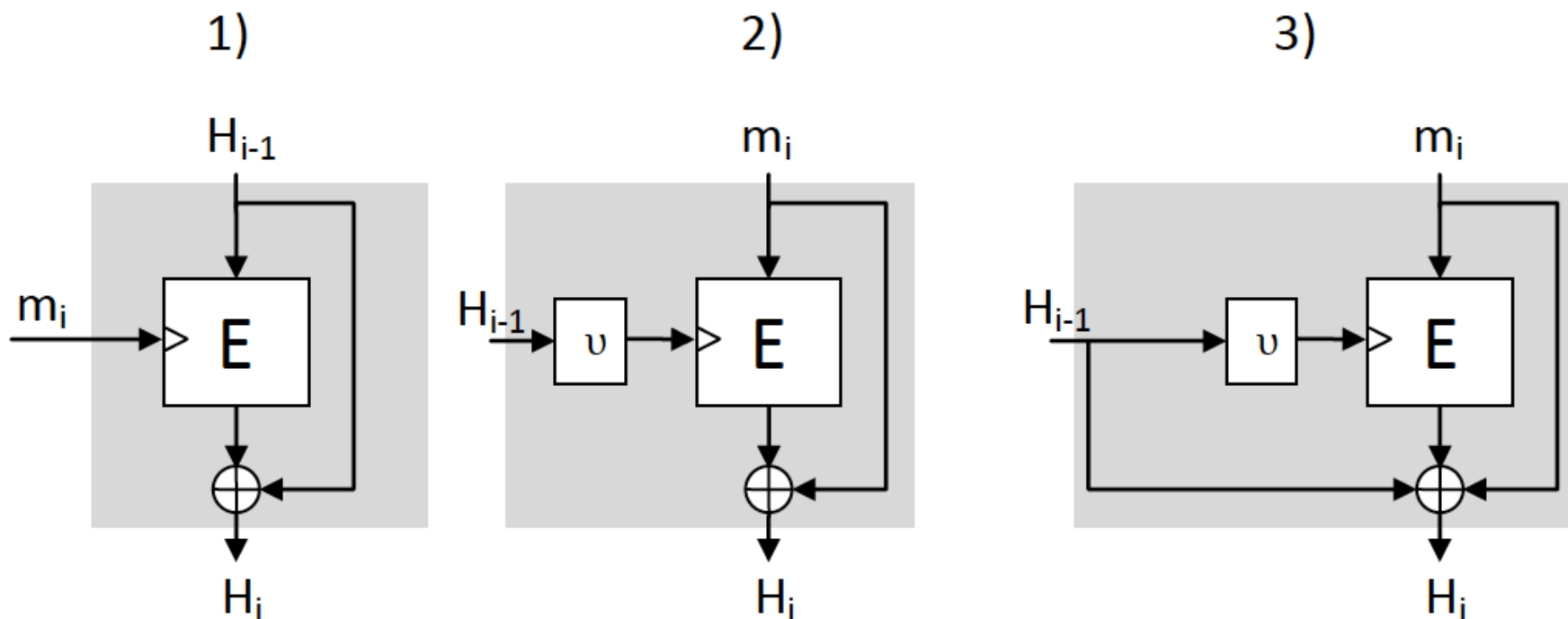
- długość bitowa wyznaczanego skrótu,
- liczba iteracji,
- liczba wykorzystywanych elementarnych funkcji logicznych,
- liczba wykorzystywanych predefiniowanych stałych,
- liczba wykorzystywanych operacji logicznych.

MDC - trzy klasy funkcji (ze względu na budowę)

- z wykorzystaniem szyfrów blokowych
- zaprojektowane wyłącznie do wyznaczania skrótu
- w konstrukcji korzysta się z arytmetyki modularnej

...

Schematy funkcji skrótu wykorzystujące szyfry blokowe

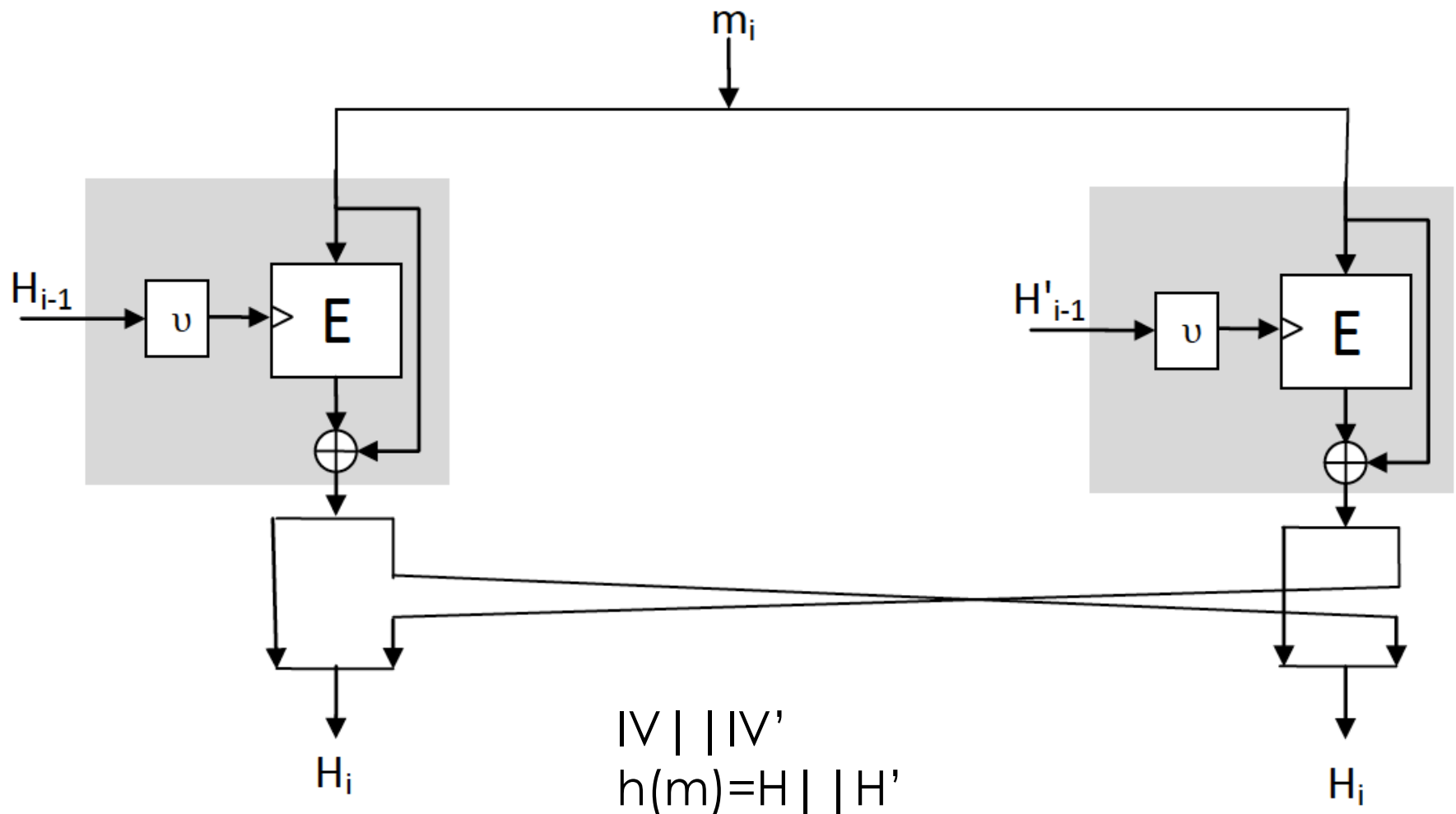


1. Davies'a - Meyera
2. Matyasa - Meyera - Oseasa
3. Miyaguchi - Preneela

Używają trzy wstępnie ustalone komponenty

- ogólny n -bitowy szyfr blokowy E_k którego parametrem jest klucz symetryczny k
- funkcji v , która odwzorowuje n -bitowe wejścia w klucze k odpowiednie do użycia z E (jeśli klucze E mają także długości n , to g może być funkcją tożsamościową)
- ustalona (zwykle n -bitowa) wartość początkowa IV , odpowiednia do użycia z E

Schemat funkcji skrótu podwójnej długości MDC-2 (ISO/IEC 10118)

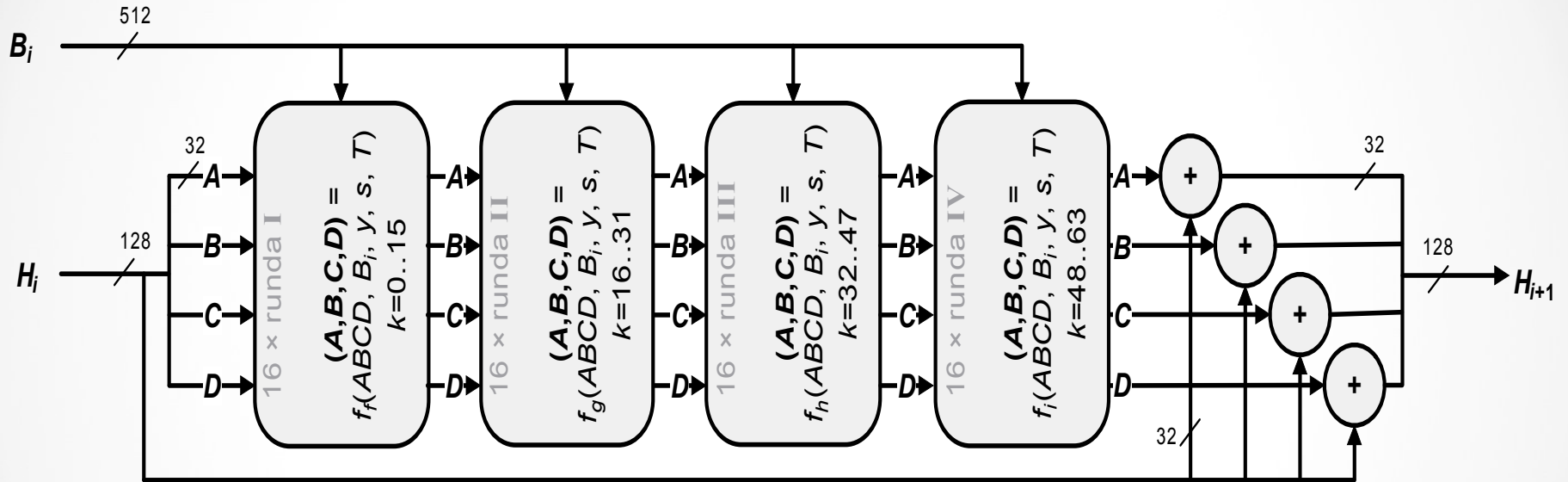


Funkcje dedykowane

Są to funkcje, które są specjalnie projektowane na potrzeby określonego celu skracania, z myślą o optymalizacji i szybkości.

przykłady: rodzina MD, SHA-1, RIPEMD- 128, 160.

MD5



F u n k c j a elementarna	Etap wykonywania	Wynik działania
$f_f(b, c, d)$	runda I, kroki $k = 0, 1, \dots, 15$	$bc \vee bd$
$f_g(b, c, d)$	runda II, kroki $k = 16, 17, \dots, 31$	$bd \vee c\bar{d}$
$f_h(b, c, d)$	runda III, kroki $k = 32, 33, \dots, 47$	$b \oplus c \oplus d$
$f_i(b, c, d)$	runda IV, kroki $k = 48, 49, \dots, 63$	$c \oplus (b \vee \bar{d})$

Funkcje skrótu wykorzystujące arytmetykę modularną

Koncepcja:

- takie zaprojektowanie iteracyjnej funkcji skrótu, aby arytmetyka mod M stanowiła podstawę funkcji kompresującej

Uzasadnienie:

- wykorzystanie istniejącego oprogramowania i sprzętu arytmetyki modularnej
- skalowalność

Przykład MASH-1, MASH-2

MASH

MASH-1 (Modular Arithmetic Secure Hash)

$$H_i = ((x_i \oplus H_{i-1}) \vee A)^2 \pmod{N} \oplus H_{i-1}$$

$A = 0xF00\dots00$

x_i : first 4 bits in every byte equal to 1111 (1010 in last byte)

output transformation that reduces output size to at most $n/2$

MASH-2: replace exponent 2 by $2^8 + 1$

security for n -bit RSA modulus:

- best known attacks: preimage in $2^{n/2}$, collision in $2^{n/4}$
- feedforward of H_{i-1} essential

Funkcje skrótu z kluczem

Są podstawą dla dwóch rodzajów algorytmów kryptograficznych:

MAC - message authentication code - kod uwierzytelniania komunikatu - uwierzytelnia komunikat i chroni integralność

PRF - pseudorandom function - tworzą wyglądające na losowe wartości o długości skrótu

MAC

Tworzy wartość - tzw. znacznik uwierzytelniania komunikatu M

$$T = \text{MAC}(K, M)$$

gdzie jest stosowany:

- dla każdego pakietu w:
 - IPsec
 - SSH
 - TLS
- standardy telefonii kom. 3G i 4G szyfrują ale nie uwierzytelniają

MAC

Algorytmy kodu uwierzytelniania wiadomości:

- MAC wykorzystujące szyfry blokowe - CBC-MAC
- MAC zaprojektowane na podstawie algorytmów MDC (klucz dołączany jest jako część wejścia)

PRF

To funkcja, która używa tajnego klucza do wybrania i zwrócenia instancji:

$$\text{PRF}(K, M)$$

gdy klucz jest tajny wynik jest nieprzewidywalny dla napastnika

- nie są używane samodzielnie, ale są częścią algorytmu kryptograficznego (klucz w szyfratorze, schematy identyfikacji wezwane-odpowiedź, 4G - uwierzytelnia kartę SIM i dostawcę usługi, TLS - do wygenerowania klucza sesyjnego)

Atak na funkcję skrótu

- Próba znalezienia M' , takiego że:
 $H(M)=H(M')$
- Atak metodą urodzin

Paradoks urodzin

- jaka jest minimalna wartość k taka, że prawdopodobieństwo, iż co najmniej dwóch ludzi w grupie k ludzi ma urodziny tego samego dnia, wynosi więcej niż 0,5?
- $P(n,k) = \Pr$ (co najmniej jeden duplikat wśród k elementów, gdzie każdy element może przyjąć jedną z n tak samo prawdopodobnych wartości pomiędzy 1 i n)

$$P(365, k) = 1 - \frac{365!}{(365 - k)! k^{365}}$$

Paradoks urodzin

- liczba ta wynosi 23 $P(365,23)=0,5073$
- Prawdopodobieństwo, że inna osoba będzie miała tę samą datę urodzenia, w konkretnej grupie, jest niewielkie
- Prawdopodobieństwo, że istnieje para osób w grupie która ma tę samą datę urodzenia jest większe niż 0,5
- przy 23 osobach mamy $(23(23-1))/2 = 253$ różnych par ludzi – stąd duże prawdopodobieństwo!

Atak na funkcję skrótu

- Oczekiwana liczba skrótów, jakie należy zbadać przed wystąpieniem koincydencji wynosi ok. $5,38 \cdot 10^9$ dla 64-bitowych skrótów
- Jeśli skrót jest 128-bitowy, to próbując znaleźć dwie wiadomości o takim samym skrócie należy wykonać średnio $2,31 \cdot 10^{19}$ prób

Atak na funkcję skrótu

- Yuwal zaproponował następującą strategię ataku metodą urodzinową
 1. źródło zamierza podpisać komunikat przez dodanie doń odpowiedniej m-bitowej sumy kontrolnej i zaszyfrowanie jej kluczem prywatnym A
 2. Przeciwnik generuje $2^{m/2}$ wersji komunikatu, z których wszystkie przekazują taką samą treść. Przygotowuje też taką liczbę komunikatów, z których wszystkie są wersjami komunikatu fałszywego, który ma zastąpić prawdziwy

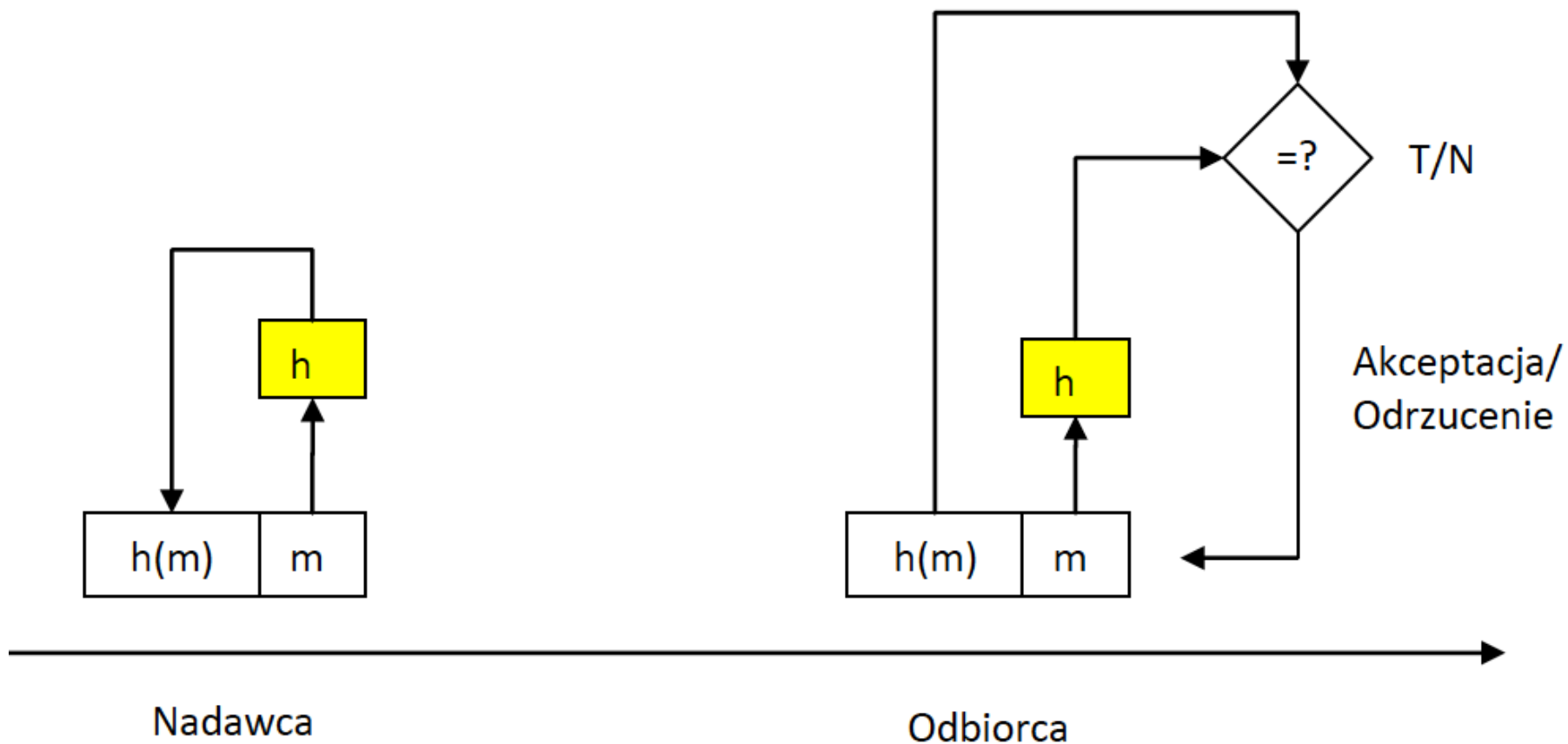
Atak na funkcję skrótu

3. Oba zbiory komunikatów zostają porównane w celu znalezienia pary, która da taki sam wynik działania funkcji haszującej. Prawdopodobieństwo sukcesu, według paradoksu dnia urodzin jest większe niż 0,5. Jeśli nie uda się znaleźć takiej pary, generuje się następne prawdziwe i fałszywe komunikaty aż do skutku.

Atak na funkcję skrótu

4. Przeciwnik przedstawia prawdziwą wersję A do sygnatury. Sygnaturę tę można dołączyć do wersji fałszywej i przesłać komunikat do adresata. Ponieważ obie wersje mają taką samą wartość funkcji haszującej, dadzą taką samą sygnaturę, przeciwnik może więc być pewien sukcesu mimo nieznajomości klucza do szyfru.

Zastosowanie funkcji skrótu - integralność danych



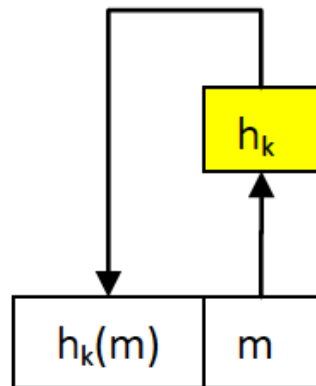
Tripwire

- stosuje funkcje skrótu do wykrywania zmian w plikach
- monitoruje prawa dostępu do plików, czasy modyfikacji plików oraz inne ważne właściwości plików (wybierane przez administratora)
- 4 tryby pracy:
 - tworzenia bazy danych
 - uaktualniania bazy danych,
 - sprawdzania integralności,
 - interaktywnego uaktualniania bazy danych.

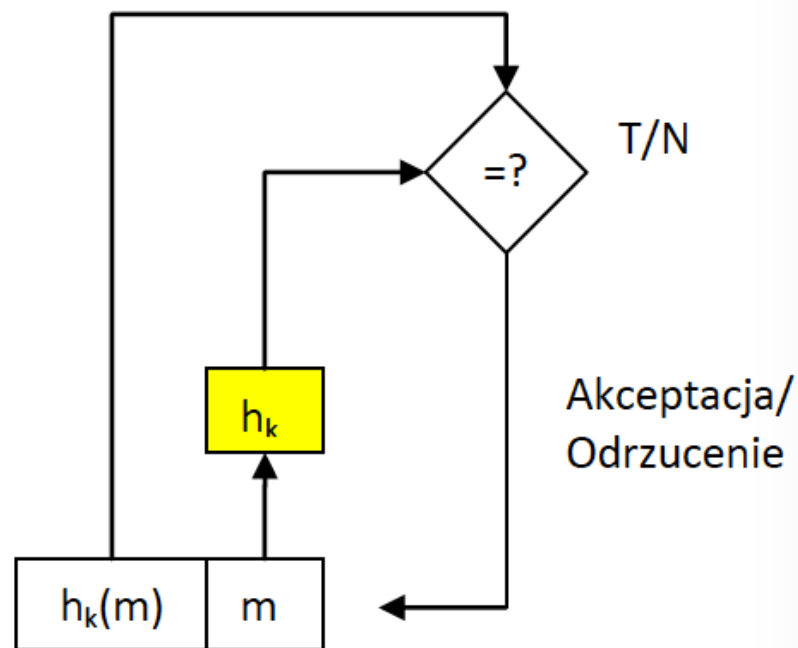
Funkcje skrótu z kluczem - HMAC

H. Krawczyk, M. Bellare, R. Canetti, RFC 2104, 1997.

Integralność danych +
Uwierzytelnianie danych



Nadawca



Odbiorca

Zastosowania

- **Usługi certyfikacyjne**

Kategoria ta składa się z protokołów certyfikacyjnych. Najważniejszym wykorzystaniem funkcji skrótu tej grupy są schematy podpisu cyfrowego. W schematach tych podpisywana nie jest sama wiadomość, ale właśnie jej skrót wyliczany za pomocą funkcji skrótu. Innym wykorzystaniem funkcji skrótu są znaczniki czasowe wykorzystywane w usługach certyfikacyjnych.

- **Uwierzytelnienie**

Grupa protokołów potwierdzająca tożsamość klienta. Jednym z przykładów jest system Kerberos, który dostarcza mechanizmy uwierzytelniania, chroni przed podsłuchiowaniem oraz zapewnia integralność danych w modelu klient-serwer. W systemie funkcje skrótu wykorzystywane są do wyliczania skrótu wprowadzanego przez klienta hasła, który to w dalszej części protokołu staje się tajnym kluczem klienta. Inne protokoły tej grupy to: IEEE802.1X-EAP czy APOP.

- **Aktualizacja oprogramowania**

Systemy aktualizacji oprogramowania (łaty, aktualizacje, sygnatury wirusów) wykorzystują funkcje skrótu w celu weryfikacji integralności oprogramowania.

Zastosowania

- **Bezpieczna komunikacja**

Grupa zawierająca w sobie protokoły wymiany klucza. Jednym z najpopularniejszych protokołów tej grupy jest szeroko stosowany IPsec. Zapewnia on bezpieczną komunikację w warstwie sieciowej poprzez szyfrowanie i uwierzytelnianie pakietów IP. Protokół wymiany klucza (IKE – Internet Key Exchange) zastosowany w IPsec używa funkcji skrótu do generacji ciągów pseudolosowych. Dodatkowo funkcja skrótu jest używana do weryfikacji integralności i uwierzytelnienia wszystkich wiadomości przesyłanych w tym protokole. Innym przykładem jest protokół SSH, który zapewnia uwierzytelnianie użytkowników i tworzy bezpieczny kanał pomiędzy lokalnym a zdalnym komputerem. SSH wykorzystuje funkcje skrótu jako kody uwierzytelniające wiadomości (MAC). To samo zastosowanie funkcji skrótu wykorzystane jest w protokole SSL/TLS.

Zastosowania

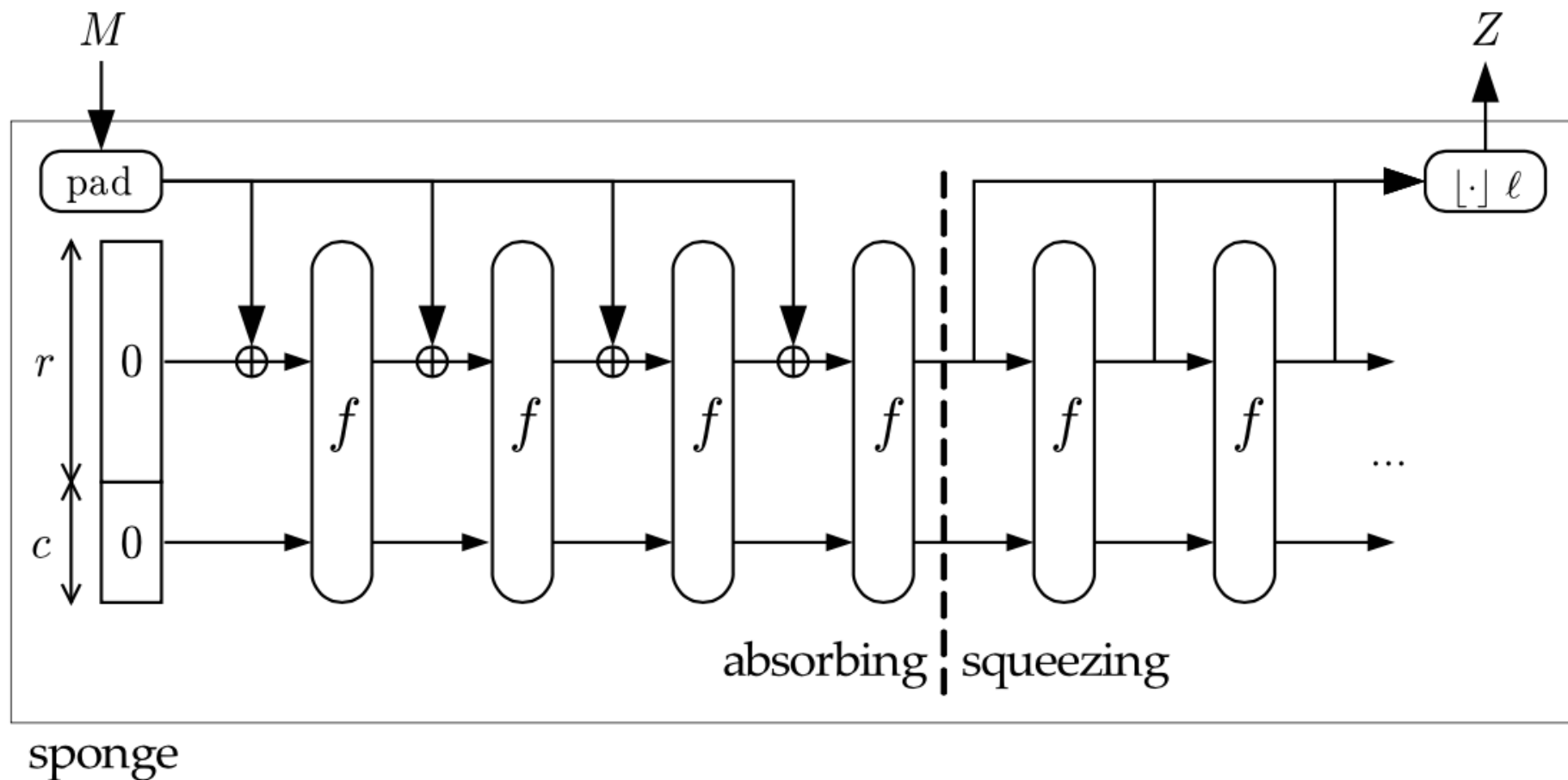
- **Bezpieczna poczta elektroniczna**

Kategoria zapewniająca bezpieczną wymianę wiadomości e-mail, ochronę prywatności i sprawdzanie integralności wiadomości. Protokół S/MIME używa funkcji skrótu w schemacie podpisu cyfrowego, zapewniając autentyczność i integralność przesyłanych wiadomości. Również jedno z najpopularniejszych narzędzi do szyfrowania poczty elektronicznej – PGP (Pretty Good Privacy), używa funkcji skrótu dla zapewnienia integralności wiadomości e-mail.

- **Przechowywanie haseł**

Większość współczesnych systemów informatycznych nie przechowuje haseł w postaci jawnej, lecz wykorzystuje funkcje skrótu do tworzenia skrótów haseł, które następnie są przechowywane w systemie. Rozwiązania takie spotykane są powszechnie w większości systemów baz danych (np. Oracle, MySQL) oraz w systemach operacyjnych (np. Windows, UNIX).

Konstrukcje typu Duplex i Sponge, 2010



KECCAK

- Instantiation of a *sponge function*
- the **permutation** KECCAK- f
 - 7 permutations: $b \in \{25, 50, 100, 200, 400, 800, 1600\}$
- Security-speed trade-offs using the same permutation, e.g.,
 - SHA-3 instance: $r = 1088$ and $c = 512$
 - permutation width: 1600
 - security strength 256: post-quantum sufficient
 - Lightweight instance: $r = 40$ and $c = 160$
 - permutation width: 200
 - security strength 80: same as SHA-1

Jak zbudować silną permutację

- Build it as is an iterated permutation
- Like a block cipher
 - Sequence of identical rounds
 - Round consists of sequence of simple step mappings
- ...but not quite
 - No key schedule
 - Round constants instead of round keys
 - Inverse permutation need not be efficient

Kryteria silnej permutacji

- Classical LC/DC criteria
 - Absence of large differential propagation probabilities
 - Absence of large input-output correlations
- Infeasibility of the CICO problem
 - Constrained Input Constrained Output
 - *Given partial input and partial output, find missing parts*
- Immunity to
 - Integral cryptanalysis
 - Algebraic attacks
 - Slide and symmetry-exploiting attacks
 - ...

KECCAK

- Instantiation of a *sponge function*
- the **permutation** KECCAK- f
 - 7 permutations: $b \in \{25, 50, 100, 200, 400, 800, 1600\}$
- Security-speed trade-offs using the same permutation, e.g.,
 - SHA-3 instance: $r = 1088$ and $c = 512$
 - permutation width: 1600
 - security strength 256: post-quantum sufficient
 - Lightweight instance: $r = 40$ and $c = 160$
 - permutation width: 200
 - security strength 80: same as SHA-1

Capacity

Rate

Strength

256

1344

128

384

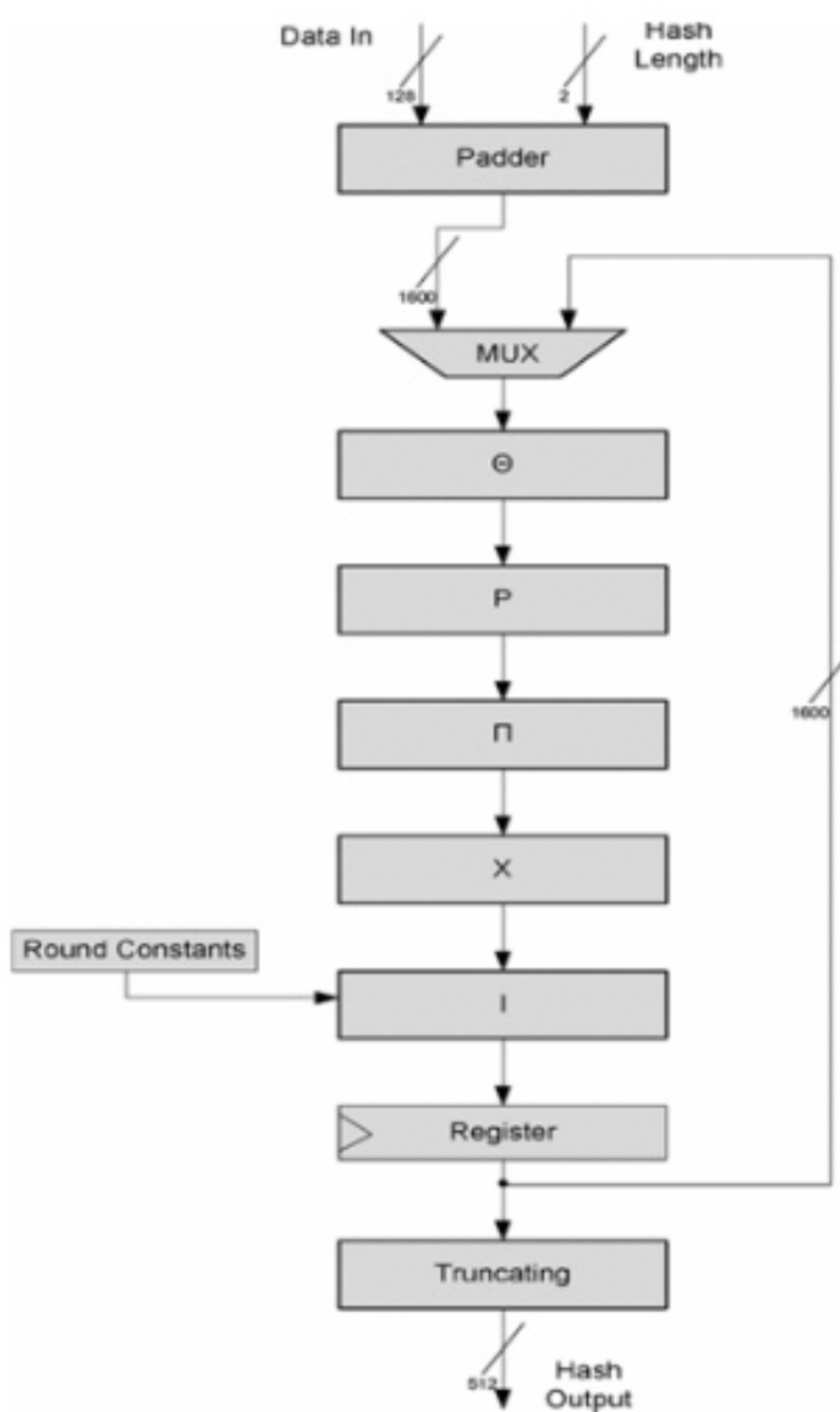
1216

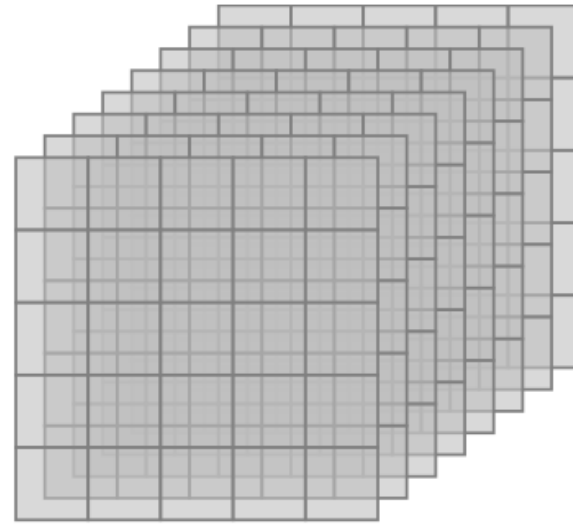
192

512

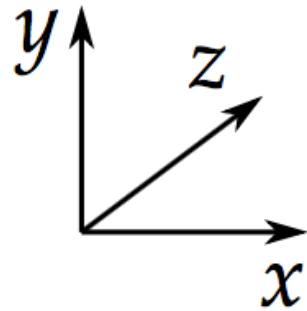
1088

256

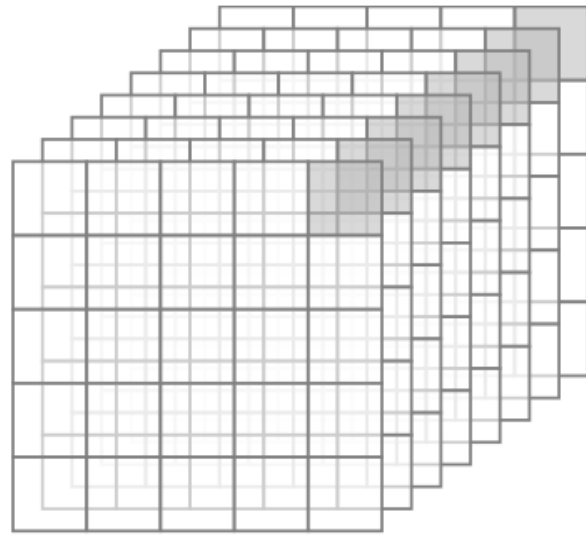




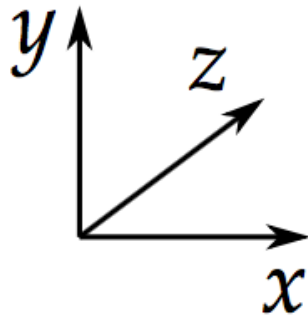
state



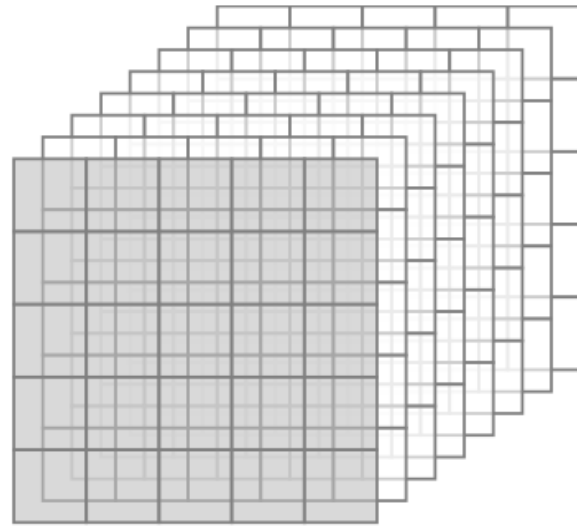
- 5×5 **lanes**, each containing 2^ℓ bits (1, 2, 4, 8, 16, 32 or 64)
- (5×5) -bit **slices**, 2^ℓ of them



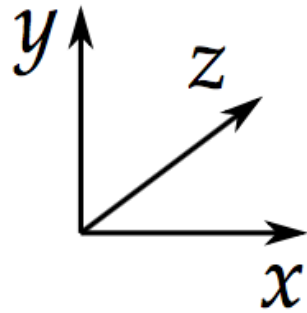
lane



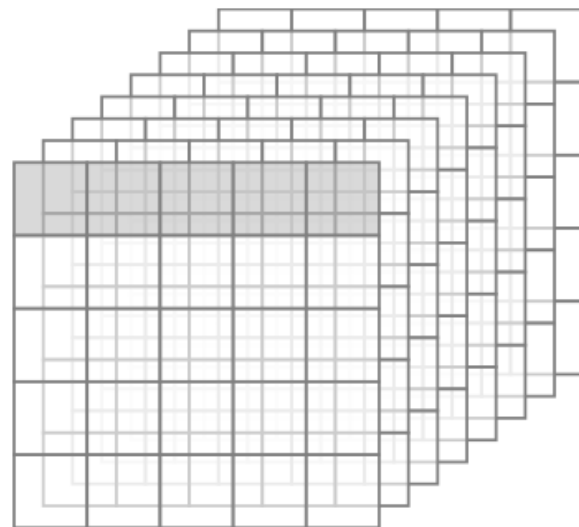
- 5×5 **lanes**, each containing 2^ℓ bits (1, 2, 4, 8, 16, 32 or 64)
- (5×5) -bit **slices**, 2^ℓ of them



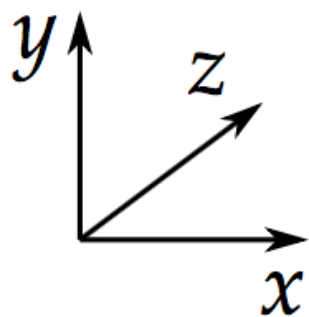
slice



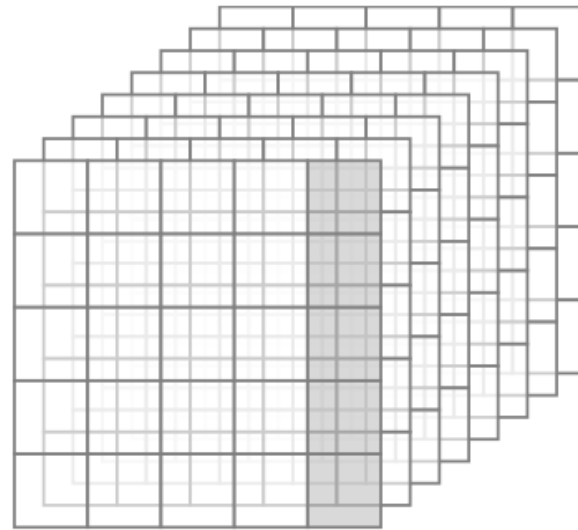
- 5×5 **lanes**, each containing 2^ℓ bits (1, 2, 4, 8, 16, 32 or 64)
- (5×5) -bit **slices**, 2^ℓ of them



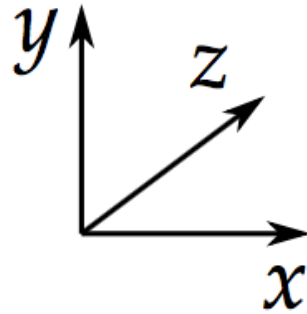
row



- 5×5 **lanes**, each containing 2^ℓ bits (1, 2, 4, 8, 16, 32 or 64)
- (5×5) -bit **slices**, 2^ℓ of them

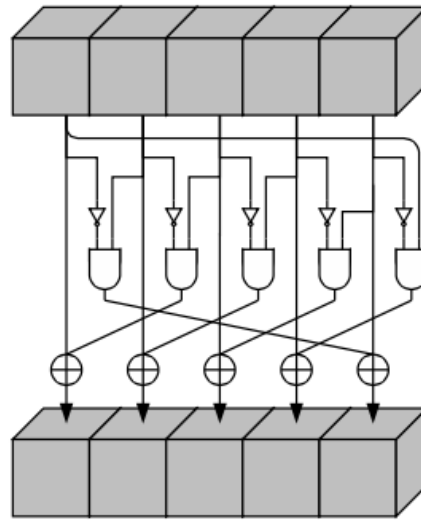


column



- 5×5 **lanes**, each containing 2^ℓ bits (1, 2, 4, 8, 16, 32 or 64)
- (5×5) -bit **slices**, 2^ℓ of them

χ , the nonlinear mapping in KECCAK- f

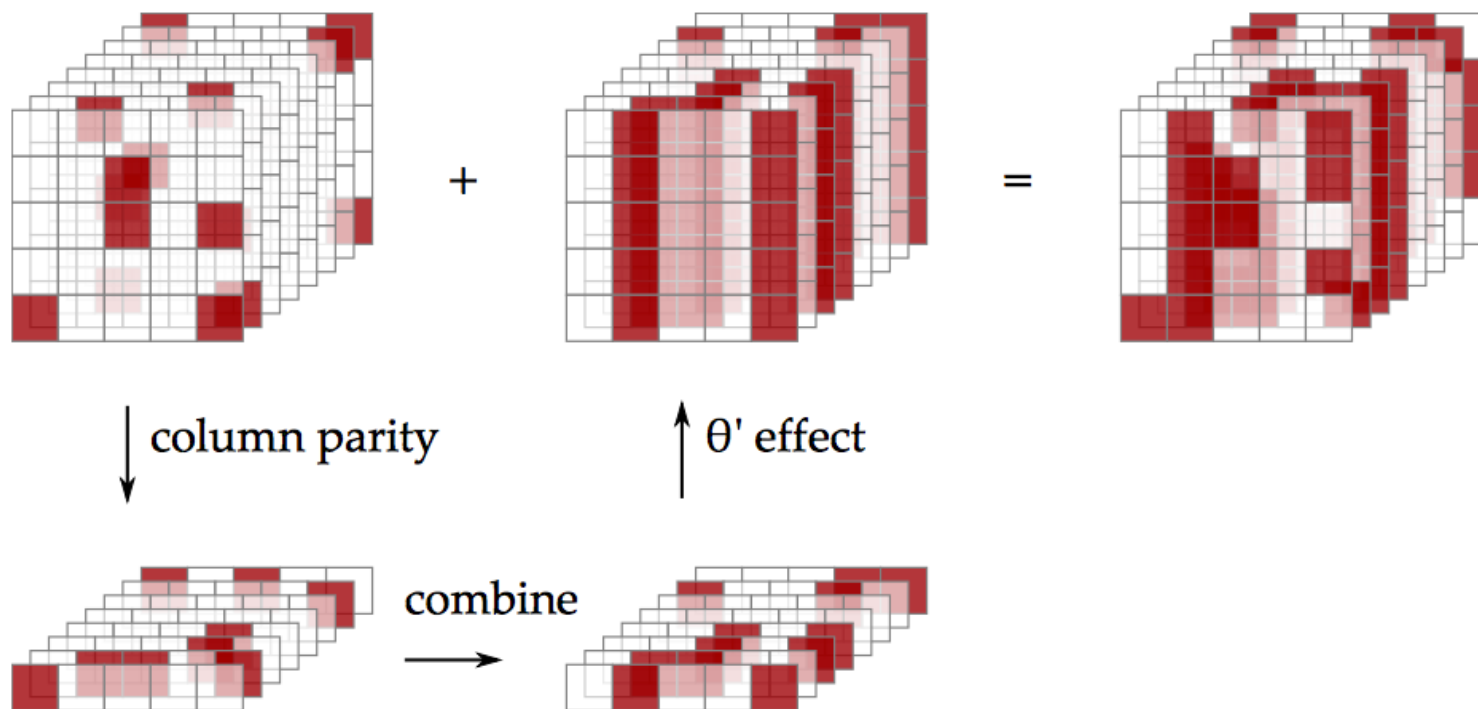


- “Flip bit if neighbors exhibit 01 pattern”
- Operates independently and in parallel on 5-bit rows
- Algebraic degree 2, inverse has degree 3
- LC/DC propagation properties easy to describe and analyze

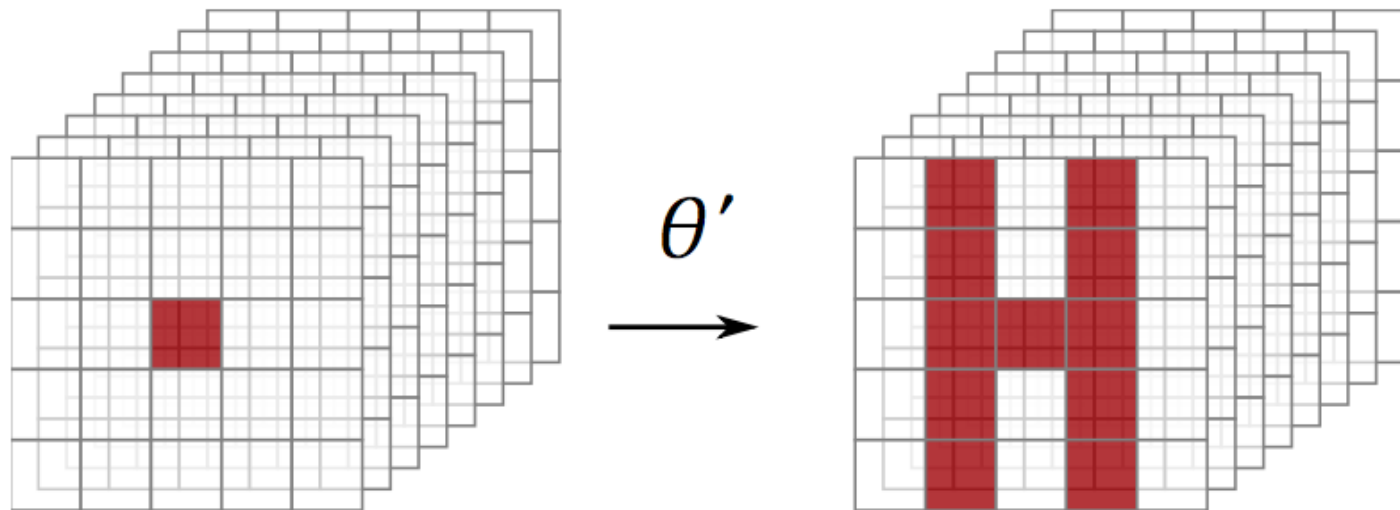
θ' , a first attempt at mixing bits

- Compute parity $c_{x,z}$ of each column
- Add to each cell parity of neighboring columns:

$$b_{x,y,z} = a_{x,y,z} \oplus c_{x-1,z} \oplus c_{x+1,z}$$



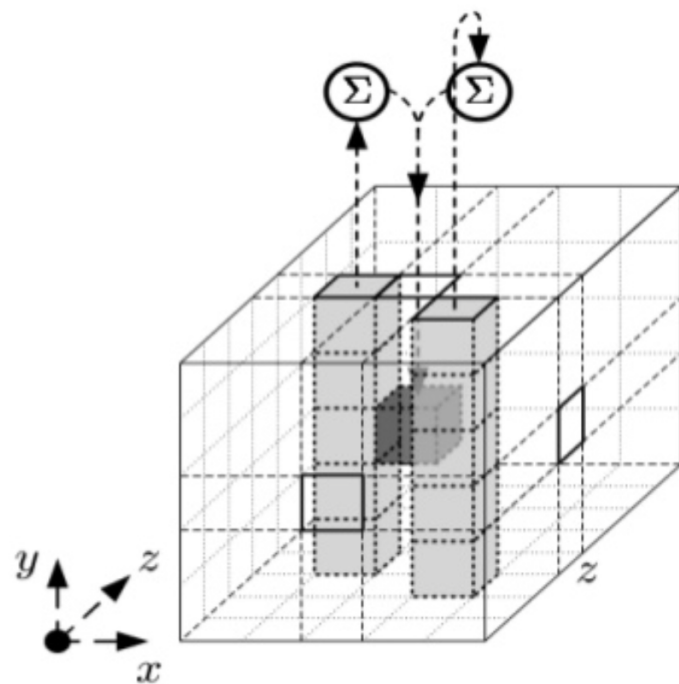
Diffusion of θ'



- Renders the internal state into a 5-by-5 array of 64-bit elements.
- Computes the parities of each column and combines them with an exclusive-or (XOR) operator.
- Then it XORs the resulting parity to each state bit as follows:

$$S[i][j][k] \wedge = \text{parity}(S[0\dots4][j-1][k]) \wedge \text{parity}(S[0\dots4][j+1][k-1])$$

where $i = 0\dots4; j = 0\dots4; k = 0\dots63$

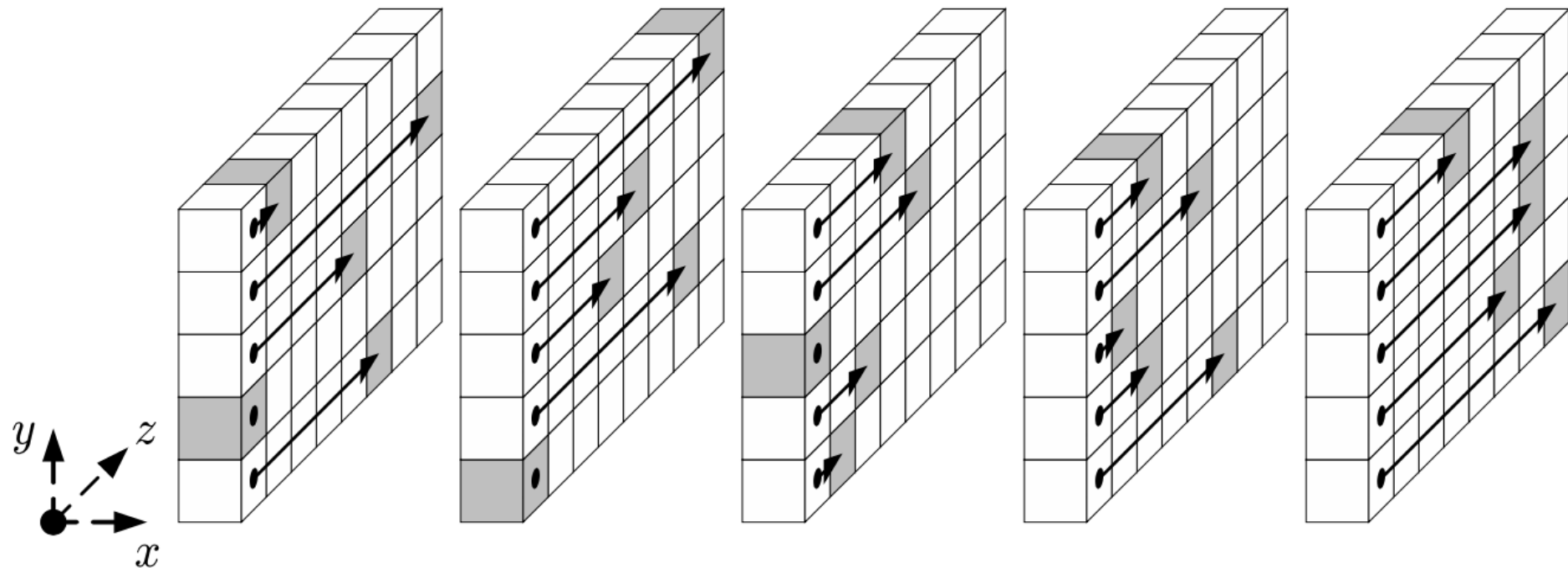


ρ for inter-slice dispersion

- We need diffusion between the slices ...
- ρ : cyclic shifts of lanes with offsets

$$i(i+1)/2 \bmod 2^\ell$$

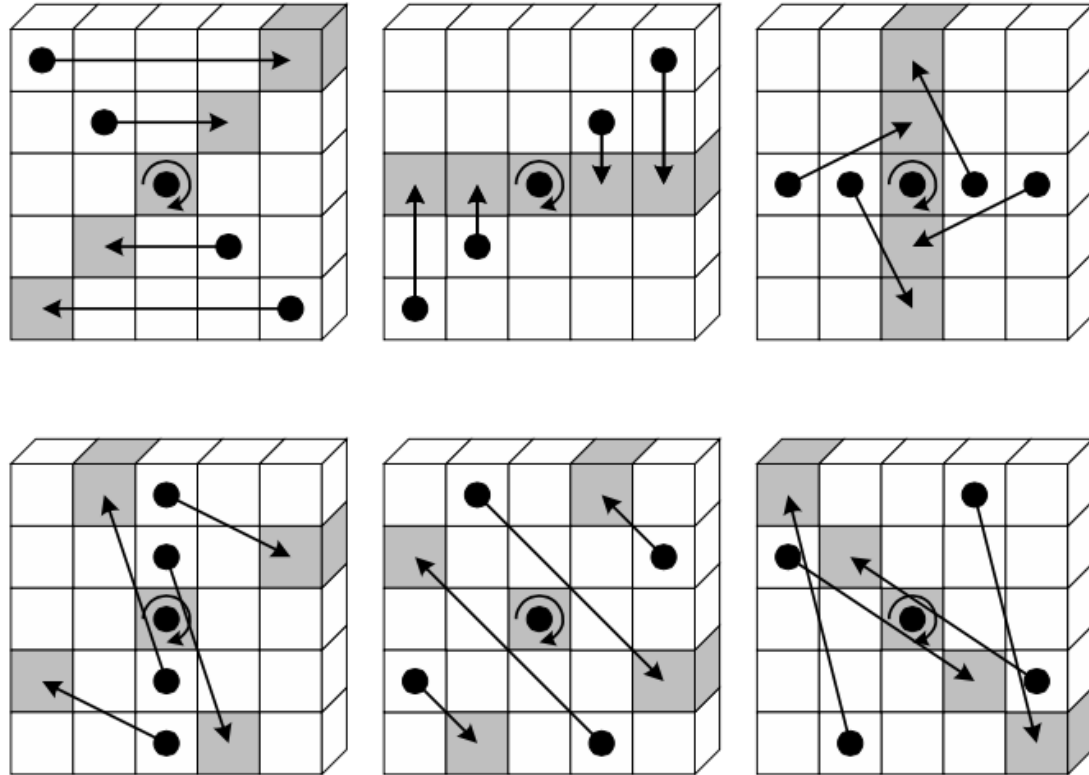
- Offsets cycle through all values below 2^ℓ



ι to break symmetry

- XOR of round-dependent constant to lane in origin
- Without ι , the round mapping would be symmetric
 - invariant to translation in the z-direction
- Without ι , all rounds would be the same
 - susceptibility to *slide* attacks
 - defective cycle structure
- Without ι , we get simple fixed points (000 and 111)

π for disturbing horizontal/vertical alignment



$$a_{x,y} \leftarrow a_{x',y'} \text{ with } \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} x' \\ y' \end{pmatrix}$$

KECCAK- f summary

- Round function:

$$R = \iota \circ \chi \circ \pi \circ \rho \circ \theta$$

- Number of rounds: $12 + 2\ell$

- KECCAK- $f[25]$ has 12 rounds
- KECCAK- $f[1600]$ has 24 rounds

- Efficiency

- high level of parallelism
- flexibility: bit-interleaving
- software: competitive on wide range of CPU
- dedicated hardware: very competitive
- suited for protection against side-channel attack

Performance in software

- Faster than SHA-2 on all modern PC
- KECCAKTREE faster than MD5 on some platforms

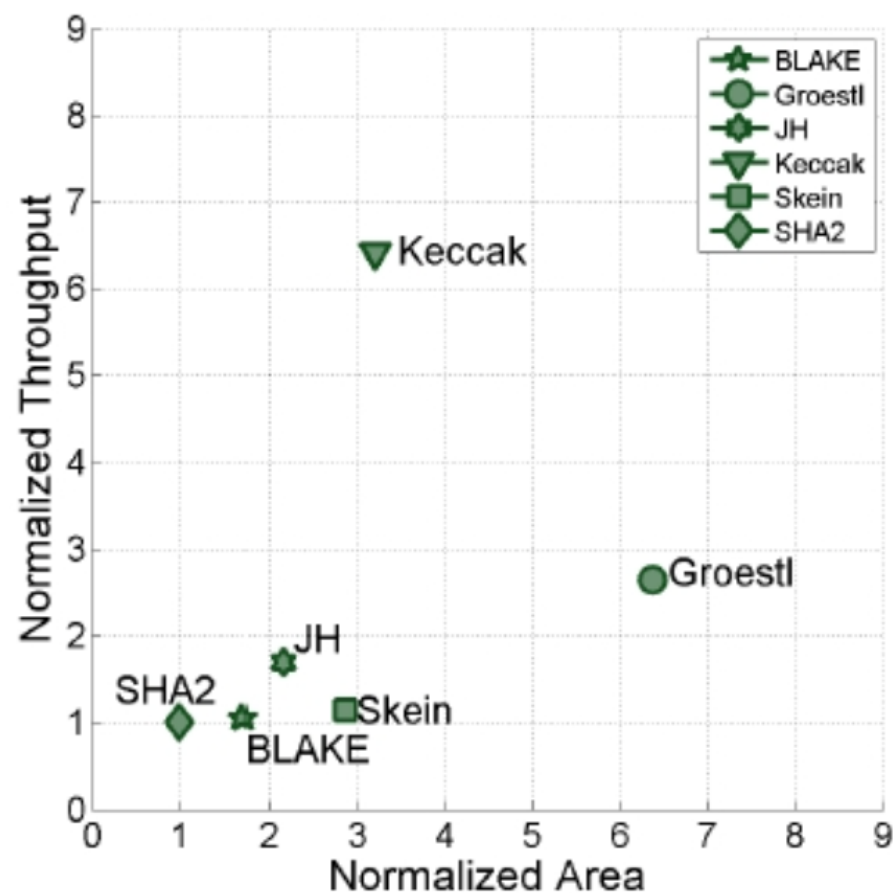
C/b	Algo	Strength
4.79	keccakc256treed2	128
4.98	md5	< 64
5.89	keccakc512treed2	256
6.09	sha1	< 80
8.25	keccakc256	128
10.02	keccakc512	256
13.73	sha512	256
21.66	sha256	128

[eBASH, hydra6, <http://bench.cr.yp.to/>]

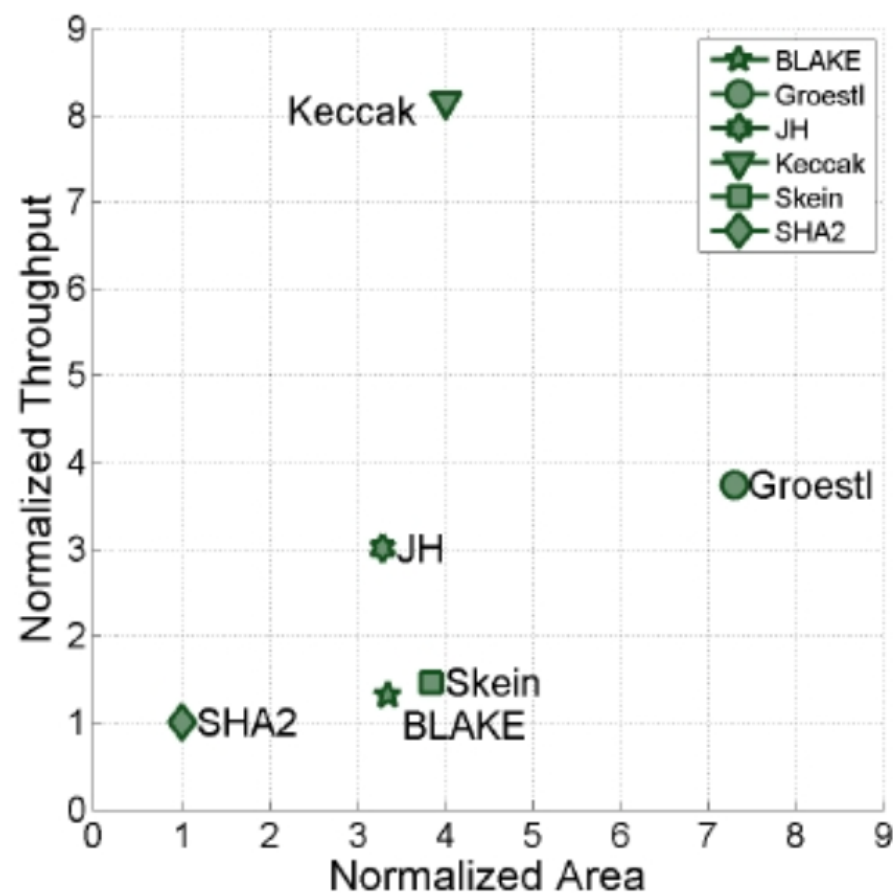
Efficient and flexible in hardware

From Kris Gaj's presentation at SHA-3, Washington 2012:

ASIC

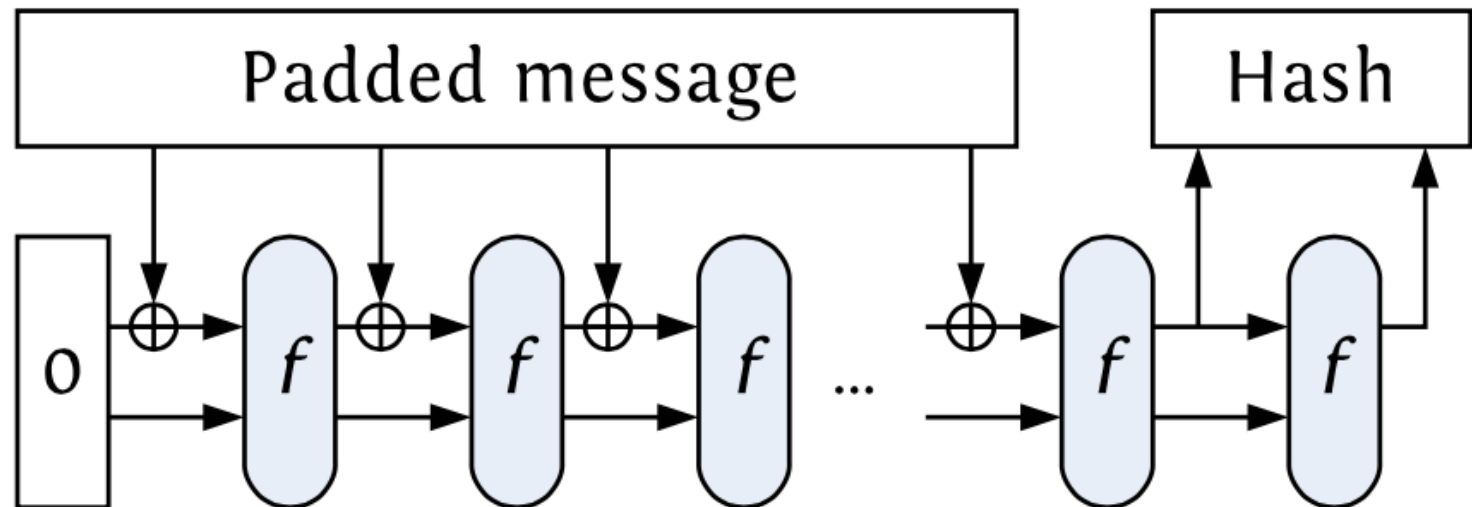


Stratix III FPGA



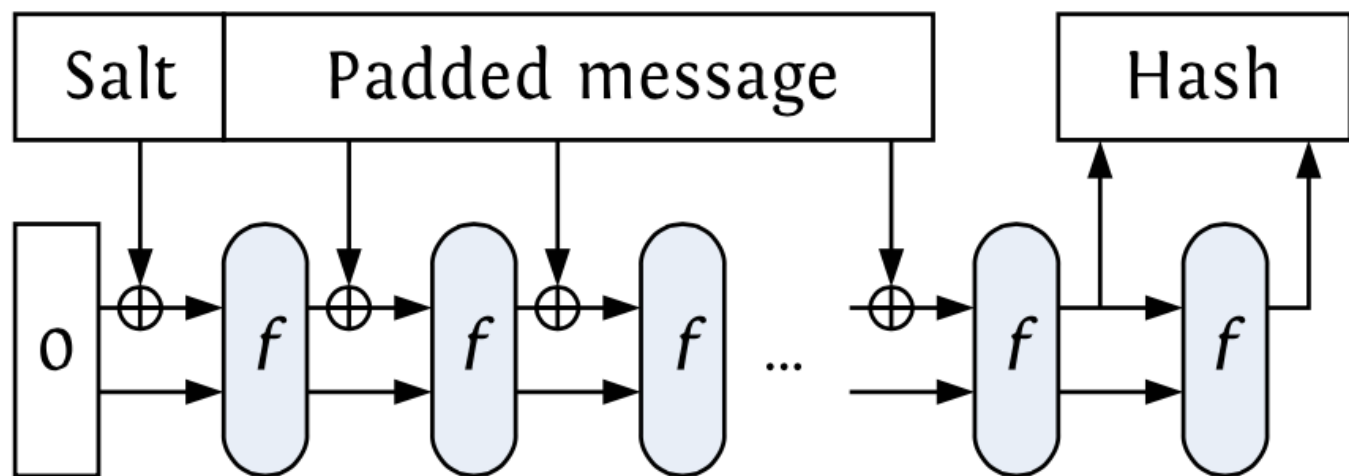
Aplikacje stosujące KECCAK

Regular hashing



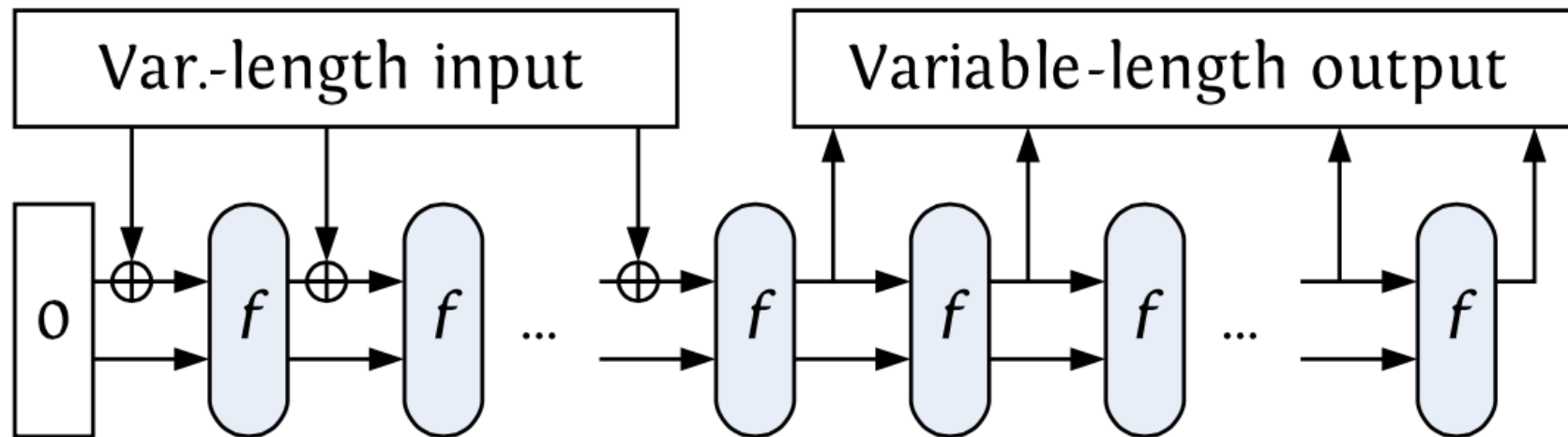
- Electronic signatures
- Data integrity (*shaXsum ...*)
- Data identifier (*Git, online anti-virus, peer-2-peer ...*)

Salted hashing



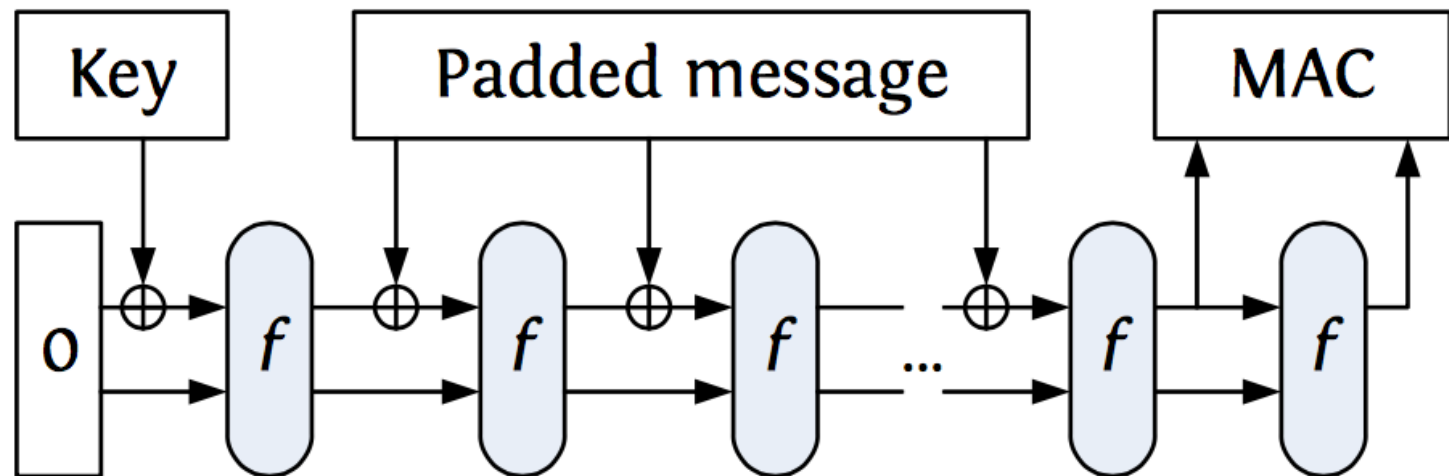
- Randomized hashing (RSASSA-PSS)
- Password storage and verification (*Kerberos*, /etc/shadow)

Mask generation function



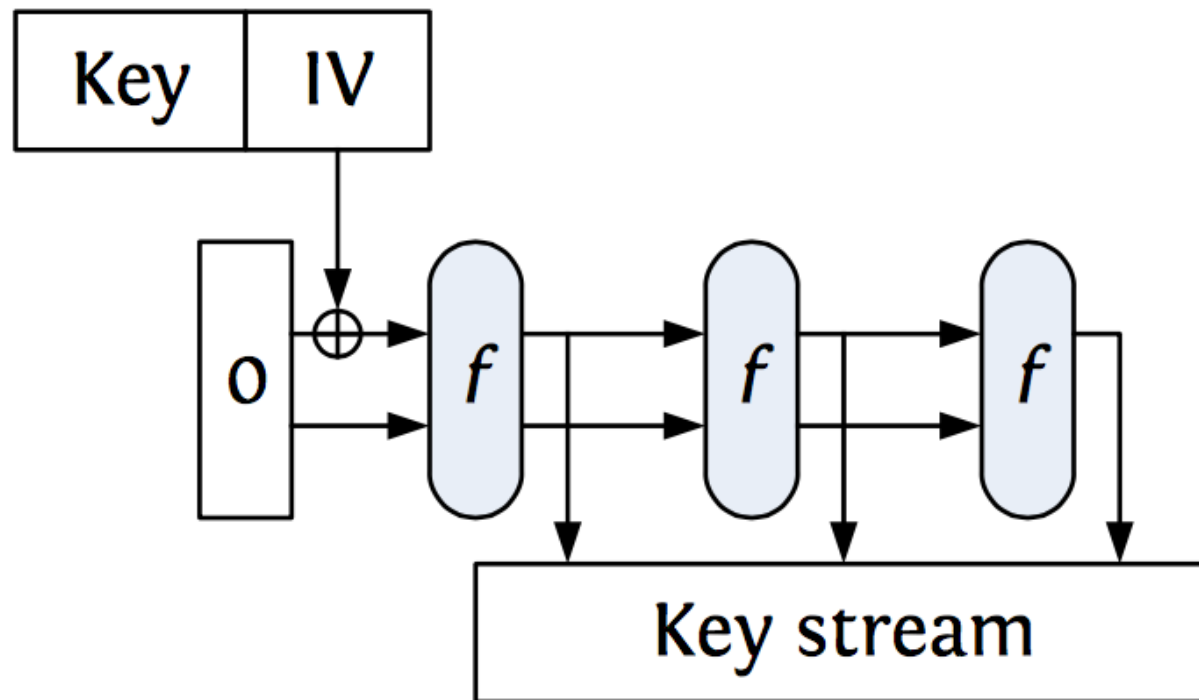
- Key derivation function in SSL, TLS
- Full-domain hashing in public key cryptography
 - electronic signatures RSASSA-PSS [PKCS#1]
 - encryption RSAES-OAEP [PKCS#1]
 - key encapsulation methods (KEM)

Message authentication codes



- As a message authentication code
- Simpler than HMAC [FIPS 198]
 - Required for SHA-1, SHA-2 due to length extension property
 - No longer needed for sponge

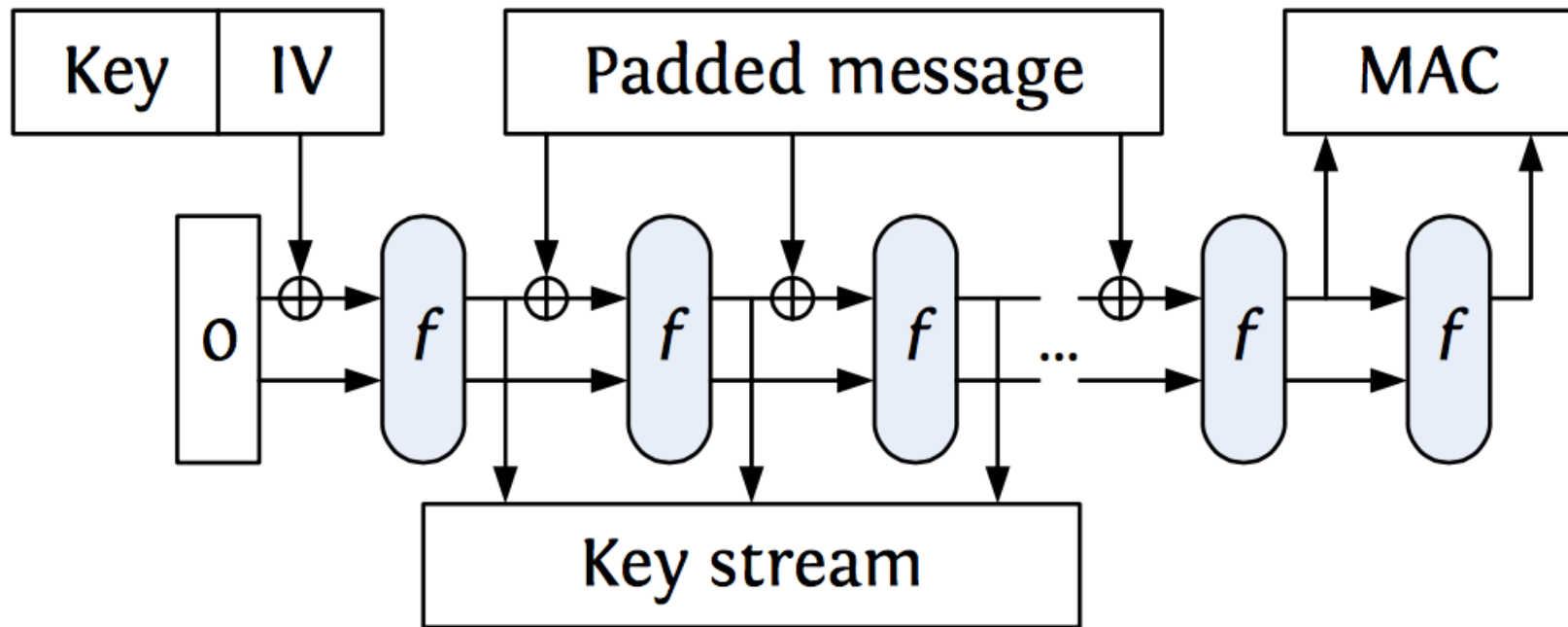
Stream encryption



- As a stream cipher

- Long output stream per IV: similar to OFB mode
- Short output stream per IV: similar to counter mode

Single pass authenticated encryption



- Authentication and encryption in a **single** pass!
- Secure messaging (*SSL/TLS, SSH, IPSEC ...*)